

PRODUCT SPECIFICATION

Modern Multi-Tenant Point of Sale System

Product Requirements, Architecture and Technology Recommendation

REFERENCE DIRECTION

Inspired by the operational breadth of SambaPOS, while maintaining an entirely original user interface, workflow presentation, branding and implementation.

Document	Details
Prepared for	Client review and project planning
Prepared by	ÀJÓKÉ CODE SPHERE
Date	19 July 2026
Market focus	Nigeria - restaurants, hospitality and retail
Status	Draft for feedback and approval

1. Executive Summary

This document defines the requirements and recommended technology architecture for a modern Point of Sale (POS) and business management platform. The solution is intended for restaurants, bars, cafés, hotels, lounges, bakeries, supermarkets, pharmacies and other retail businesses, with strong support for Nigerian operating conditions.

The platform will combine fast touchscreen sales, restaurant table management, kitchen and bar order routing, inventory, purchasing, customers, suppliers, payments, cash shifts, analytics and multi-branch administration. It will be delivered as a secure multi-tenant product so that each subscribing business operates in an isolated workspace.

PRIMARY RECOMMENDATION

Use TypeScript across the frontend, desktop terminal, backend and optional mobile application. Combine React, Electron, Node.js, Express, PostgreSQL, Prisma, SQLite and Socket.IO. Start with a modular monolith and introduce Redis only when scale or background processing requires it.

2. Product Objectives

- Provide a fast and intuitive sales experience optimized for touchscreens, tablets, laptops and desktop POS terminals.
- Support restaurant, hospitality and retail workflows from a shared configurable platform.
- Continue essential sales operations during unstable or unavailable internet connectivity.
- Give owners real-time visibility into sales, stock, employees, branches, cash and profitability.
- Protect financial records through permissions, approvals, audit trails and reliable transaction controls.
- Allow multiple independent businesses to subscribe without exposing one tenant's information to another.
- Localize currency, payment integrations, receipt sharing and operational patterns for Nigeria.

3. Target Businesses

Hospitality	Retail and Services
Restaurants, cafés and quick-service outlets	Supermarkets, minimarts and convenience stores
Bars, lounges and clubs	Pharmacies and health-product retailers
Hotels and guest houses	Bakeries and specialty stores
Food courts and multi-station kitchens	General retail and service counters

4. Users, Roles and Permissions

Role	Primary Responsibilities
Platform Super Administrator	Manage tenants, subscription plans, platform settings, support and system-wide monitoring.

Role	Primary Responsibilities
Business Owner	View all authorized branches, configure the business and review financial performance.
Branch Manager	Manage branch operations, approvals, staff, stock, cash reconciliation and reports.
Cashier	Create sales, receive payments, print receipts and manage an assigned register.
Waiter / Server	Open table orders, add items, submit tickets, transfer tables and request bills.
Kitchen Staff	Accept, prepare and complete food orders routed to the kitchen station.
Bar Staff	Prepare and complete drink orders routed to the bar station.
Inventory Officer	Receive, transfer, count, adjust and monitor stock.
Accountant	Review revenue, expenses, payments, tax summaries, receivables and payables.
Auditor	Read reports, transaction history and audit trails without operational editing rights.

Permissions must be configurable by tenant, branch and action. Sensitive actions such as refunds, large discounts, voids, stock adjustments and register closures should optionally require a manager PIN or approval.

5. Multi-Tenant and Multi-Branch Architecture

Each registered business will operate inside a logically isolated tenant workspace. Every business record must carry a tenant identifier, and tenant-aware authorization must be enforced in the API and database access layer.

- Separate products, menus, customers, suppliers, employees, branches, terminals, transactions and reports.
- Tenant-specific logo, colours, receipt information, taxes, service charges, currency and business rules.
- Branch-specific stock, pricing, users, registers, printers, kitchen stations and reporting.
- Subscription limits for branches, users, terminals, features, integrations and storage.
- No cross-tenant access, including through reports, exports, URLs, background jobs or synchronization.

6. POS Sales Terminal

The POS terminal should prioritize speed, accuracy and minimal interaction. The main screen should place product categories and search beside a persistent current-order panel.

- Product categories, image cards, search, SKU lookup and barcode scanning.
- Add items, change quantity, attach notes, select variants, modifiers and add-ons.
- Calculate configurable tax, service charges, line discounts and order discounts.
- Assign a customer, customer group, price list, loyalty profile or account balance.
- Hold and retrieve orders; save quotations; duplicate approved orders.

- Split or merge orders and accept multiple payment methods on one sale.
- Void, refund or mark complimentary items subject to permissions and audit logging.
- Complete payment, open the cash drawer, print a receipt or share a digital receipt.
- Provide keyboard shortcuts and large touchscreen controls for experienced and new cashiers.

7. Restaurant Floor and Table Management

- Visual floor-plan editor supporting multiple dining areas and customizable table placement.
- Clear table states: available, occupied, reserved, awaiting payment, delayed and unavailable.
- Open orders against tables, record number of guests and assign the responsible waiter.
- Transfer items or complete orders between tables; merge or split tables where authorized.
- Split bills by selected item, quantity, percentage, equal share or individual guest.
- Track table duration, reservations, customer details and special instructions.
- Apply service charges and print a preliminary bill before final payment.

8. Kitchen and Bar Display Systems

- Separate kitchen and bar views, with configurable routing by product or preparation station.
- Order states: new, accepted, preparing, ready, served and cancelled.
- Group orders by ticket, table, waiter, service type and preparation station.
- Display modifiers, special instructions, elapsed time, priority and delay warnings.
- Allow staff to update individual items or a complete ticket.
- Notify the waiter or POS terminal when items are ready.
- Support kitchen order ticket printing and station-specific printers as an alternative to displays.
- Maintain a searchable order history and performance timing information.

9. Product, Menu and Pricing Management

- Categories, products, images, SKU, barcode, variants and units of measurement.
- Selling price, cost price, tax category, branch-specific prices and customer-group prices.
- Recipes, ingredients, yields, modifiers, add-ons, combos and product bundles.
- Scheduled availability, happy-hour pricing and branch-specific menus.
- Low-stock thresholds and rules preventing or warning against sale of unavailable products.
- Bulk import and export through validated Excel or CSV templates.

10. Inventory and Stock Control

- Current stock, available stock, committed quantities and stock movement history.
- Stock receipts, issues, branch transfers, adjustments, returns and physical counts.
- Damaged, expired or wasted stock with reason codes and approvals.
- Purchase orders, goods received notes, supplier invoices and associated stock updates.
- Reorder levels, low-stock notifications, batch tracking and expiry tracking where required.
- Inventory valuation and cost history.
- Automatic ingredient deduction based on restaurant recipes and completed sales.

11. Customer, Loyalty and Promotion Management

- Customer profile, contact information, purchase history, notes and favourite products.

- Customer groups, negotiated prices, discounts, credit limits and outstanding balances.
- Loyalty points, rewards, vouchers, gift cards, coupons and promotional rules.
- Digital receipts, account statements and transaction history.
- Controls preventing unauthorized credit sales or credit-limit overrides.

12. Supplier and Purchasing Management

- Supplier profiles, contacts, terms, opening balances and statements.
- Purchase requisitions or orders, approval workflow and expected delivery dates.
- Goods received, supplier invoices, payments, returns and outstanding balances.
- Purchase history, product cost changes and supplier cost comparisons.

13. Payments, Cash Registers and Shifts

13.1 Payment Methods

- Cash, debit or credit card, bank transfer, POS terminal payment and mobile money.
- Customer credit, gift card, voucher, split payment and tenant-defined payment methods.
- Optional integrations with Paystack, Flutterwave or Monnify using securely stored tenant credentials.
- Payment status, reference, reconciliation status and refund history.

13.2 Registers and Shifts

- Open register with an opening balance and assigned cashier.
- Cash-in, cash-out, paid-in, paid-out and reason tracking.
- Shift handover, expected cash, counted cash and variance.
- End-of-day reconciliation, register closure, X report and Z report.
- Manager approval for variances and other sensitive register actions.

14. Expenses and Financial Controls

- Expense categories, recording, supporting notes, recurrence and approval.
- Petty cash, other income, accounts receivable and accounts payable.
- Tax and service-charge summaries, daily financial summary and reconciliation.
- Gross profit and profit-and-loss summaries based on authorized cost information.
- Export-ready data for external accounting systems.

15. Reports and Analytics

Reporting Area	Examples
Sales	Daily, weekly, monthly and annual sales; branch, terminal, cashier, waiter, product and category.
Payments	Payment-method breakdown, pending references, refunds and reconciliation.
Controls	Discounts, voids, complimentary items, returns and manager approvals.
Profitability	Gross profit, margin, cost trends and category contribution.

Reporting Area	Examples
Inventory	Stock valuation, movements, low stock, out of stock, expiry and wastage.
Customers and suppliers	Purchases, balances, loyalty activity, supplier purchases and liabilities.
Restaurant operations	Table turnover, peak hours, kitchen preparation time and delayed tickets.
Cash	Register activity, expected versus actual cash and cashier variance.

All applicable reports should support date, branch, user and status filters, with export to PDF, Excel or CSV. Access to cost and profit information must be permission-controlled.

16. Receipts, Printers and POS Hardware

- Thermal receipt printers, A4 invoices, kitchen and bar ticket printers.
- Barcode scanners, label printers, cash drawers and optional customer-facing displays.
- Bluetooth, USB, LAN and Wi-Fi printer support according to the target operating system.
- Configurable receipt templates with logo, address, tax identifiers, QR code and footer.
- Reprint controls and complete reprint audit history.
- Email, downloadable PDF and WhatsApp-ready digital receipt sharing.

17. Offline Operation and Synchronization

Offline capability must be designed into the transaction model from the beginning, even if full synchronization is implemented after the stable online workflow. A desktop terminal should maintain a secure local SQLite database and a durable synchronization queue.

- Create orders, manage tables, record permitted payments and print receipts while offline.
- Route tickets to locally accessible kitchen or bar printers and displays.
- Assign globally unique transaction identifiers at the terminal.
- Upload queued records automatically when connectivity returns.
- Use idempotency controls so the same transaction cannot be posted twice.
- Display connection, last synchronization and pending-record status.
- Record synchronization failures and conflicts for controlled retry or manager review.
- Prevent server-side edits from silently overwriting newer terminal activity.

18. Security and Audit Requirements

- Secure password login and quick PIN login for authorized terminal users.
- JWT access and refresh tokens, secure session controls and device management.
- Role-, branch- and action-based permissions with default-deny behavior.
- Optional two-factor authentication for owners and administrators.
- Encryption for sensitive data in transit and appropriate protection at rest.
- Rate limiting, failed-login protection, input validation and secure headers.

- Immutable or tightly controlled audit records for sales, refunds, voids, discounts, stock changes, cash closures and configuration changes.
- Automated backups, tested restoration procedure and retention rules.
- No storage of raw card data; payment integrations should use approved gateway flows.

19. Nigerian Localization

- Nigerian Naira display and configurable currency formatting.
- Configurable VAT, taxes and service charges; tax rates must never be hard-coded.
- Business registration and tax identification fields on profiles and receipts.
- Paystack, Flutterwave and Monnify integration readiness.
- English as the default language with architecture ready for additional languages.
- WhatsApp-ready receipt sharing and workflows suitable for unstable connectivity.
- Configurable date, time, time zone, numbering and receipt formats.

20. Subscription and SaaS Administration

Plan Example	Typical Positioning
Free Trial	Time-limited evaluation with controlled features and data-retention rules.
Starter	Single outlet with essential POS and basic reporting.
Business	Inventory, more users and enhanced reporting.
Professional	Multiple branches, kitchen displays, integrations and advanced analytics.
Enterprise	Negotiated limits, support, integrations and deployment requirements.

- Limit branches, users, terminals, advanced reports, integrations, storage and backups by plan.
- Manage trials, renewal, upgrades, downgrades, billing history and subscription invoices.
- Apply a controlled grace period and account restrictions without corrupting business data.
- Provide a platform dashboard for tenant support, subscription status and operational monitoring.

21. Required Interfaces

Operational Interfaces	Management Interfaces
Public landing page and plan comparison	Business registration and setup wizard
Password login and terminal PIN login	Business-owner and branch dashboards
POS sales terminal and payment interface	Restaurant floor, tables and reservations
Kitchen and bar display systems	Products, categories, menus, recipes and modifiers
Inventory, counts, movements and transfers	Suppliers, purchases and goods received
Customers, loyalty, gift cards and promotions	Expenses, registers and staff shifts

Operational Interfaces	Management Interfaces
Employees, roles and permissions	Branches, devices, terminals and printers
Reports, analytics and exports	Receipt designer and business settings
Subscription, billing and payment history	Audit logs and platform administration

22. UI/UX Direction

- Create an original visual identity; SambaPOS is a functional reference only.
- Use large touch targets, clear hierarchy and minimal steps for high-frequency sales actions.
- Provide responsive desktop, tablet and mobile layouts where each form factor is appropriate.
- Support light and dark themes with accessible colour contrast.
- Design clear loading, empty, offline, synchronized, success, warning and error states.
- Use consistent confirmation and manager-approval patterns for sensitive actions.
- Prioritize speed in POS screens and information clarity in administration screens.

23. Recommended Technology Stack

The recommended stack matches the development team's current experience with React, TypeScript, Vite, Node.js, Express, JWT, React Native and relational databases. It minimizes language switching and supports reuse of types, validation rules and UI components.

Layer	Recommended Technology	Purpose
Primary language	TypeScript	One strongly typed language across web, desktop, server and mobile.
Web frontend	React + Vite	Administration, reports, setup and browser-based operational views.
Styling	Tailwind CSS	Reusable responsive interface system and rapid UI development.
Desktop terminal	Electron + React	Windows POS application and access to local hardware services.
Backend API	Node.js + Express + TypeScript	Business logic, authorization, integrations and tenant-aware APIs.
Primary database	PostgreSQL	Reliable relational transactions, reporting and multi-tenant records.
ORM	Prisma	Typed data access, migrations and consistent database models.
Offline database	SQLite	Durable local terminal transactions and synchronization queue.
Real-time events	Socket.IO	Live tickets, kitchen status, notifications and terminal updates.
Validation	Zod	Shared runtime validation for forms, APIs and synchronization payloads.
Caching / jobs	Redis - later phase	Caching, queues, locks and background jobs when justified by scale.
Mobile	React Native + Expo	Optional owner dashboard, waiter or stock-counting application.
Deployment	Docker + managed hosting or VPS	Consistent environments and controlled deployment.

24. Programming Languages

Language	Use
TypeScript	React interfaces, Electron desktop terminal, Node.js backend and optional React Native application.

Language	Use
SQL	Database design, PostgreSQL queries, reconciliation, reporting and performance analysis.
HTML and CSS	React interface structure and styling, primarily implemented through reusable components and Tailwind CSS.

SCOPE CONTROL

PHP, Java, C#, Python and Kotlin are not required for the first version. They should only be introduced when a specific integration or operational requirement provides a clear benefit.

25. Why Electron Is Recommended

A browser-only POS can face restrictions when communicating with thermal printers, cash drawers, customer displays and other local devices. Electron allows the team to reuse React and TypeScript while adding a controlled desktop layer for local device integration, SQLite storage, automatic updates and background synchronization.

The administration portal should remain a normal web application. Kitchen displays can be browser-based progressive web applications where local printer requirements do not demand a desktop wrapper.

26. PostgreSQL Versus MySQL

Consideration	PostgreSQL	MySQL
Team familiarity	Requires a small learning transition	Already familiar and quick to begin
Complex reporting	Excellent analytical and SQL capabilities	Capable for normal POS reporting
Data types and configuration	Strong JSON and advanced data support	Good JSON support
Constraints and financial integrity	Strong fit for rigorous relational rules	Strong when carefully designed
Recommendation	Preferred for this new multi-branch SaaS POS	Acceptable when immediate speed outweighs migration

PostgreSQL is recommended for the new platform because the system will contain highly related sales, payment, inventory and accounting records. Prisma will reduce the practical difference for most application development tasks.

27. Proposed Solution Architecture

Application	Connection and Responsibility
React Admin Portal	Connects to the API for configuration, reporting, staff, stock and multi-branch management.

Application	Connection and Responsibility
Electron POS Terminal	Uses the API when online, SQLite when offline and a controlled local hardware bridge.
Kitchen / Bar Display	Receives real-time tickets and status events through Socket.IO.
Optional Expo App	Uses the same authenticated API for selected mobile workflows.
Node.js API	Enforces business rules, permissions, tenant isolation and integrations.
PostgreSQL	Stores authoritative tenant, product, sale, payment, stock and audit records.

28. Recommended Project Structure

```

pos-platform/
  apps/
    admin-web/      React + Vite
    pos-desktop/    Electron + React
    kitchen-display/ React PWA
    mobile/         Expo, when required
  server/
    api/            Express + TypeScript
  packages/
    ui/             Shared components
    types/          Shared TypeScript types
    validation/     Shared Zod schemas
  database/
    prisma/         Schema and migrations
    seeds/          Demo and setup data

```

29. Architecture Strategy

Begin with a modular monolith: one deployable API organized into clearly separated business domains such as tenants, branches, users, catalog, orders, payments, inventory, purchasing, registers, reports and subscriptions. This keeps development and deployment manageable while preserving clear module boundaries.

- Avoid microservices in the first release.
- Share TypeScript types and Zod schemas where doing so does not expose server-only information.
- Use database transactions for payment, stock and cash operations that must succeed or fail together.
- Create a versioned API and migration strategy before pilot deployment.
- Introduce queues, caching and separate services only after measured operational need.

30. Recommended Implementation Roadmap

Phase	Scope
Foundation	Tenant model, business setup, branches, users, authentication, roles and audit foundation.
Catalog	Products, categories, modifiers, tax, price lists and import tools.
Core POS	Online sales, payments, receipts, registers and controlled refunds.
Hospitality	Floors, tables, waiters, reservations, kitchen/bar routing and displays.
Inventory	Stock ledger, purchasing, transfers, counts, recipes and wastage.
Management	Expenses, customers, loyalty, suppliers, reports and reconciliation.
Offline	SQLite storage, synchronization queue, idempotency, conflicts and recovery.
SaaS	Plans, billing, tenant limits, support tools and platform administration.
Extensions	Mobile workflows, additional integrations and advanced analytics.

31. Testing and Quality Requirements

- Unit tests for prices, tax, discounts, totals, stock deductions and permission rules.
- Integration tests for complete sale, refund, stock transfer, cash closure and synchronization workflows.
- Tenant-isolation tests that deliberately attempt cross-tenant access.
- Offline, reconnect, duplicate-message and interrupted-synchronization tests.
- Printer and hardware testing against the devices selected for deployment.
- Performance testing for peak concurrent orders and kitchen updates.
- Backup restoration, audit integrity and deployment rollback testing.
- User acceptance testing with owners, managers, cashiers, waiters and kitchen staff.

32. Final Deliverables

- Approved product requirements and workflow specification.
- Original responsive UI/UX designs and reusable design system.
- Database schema, API structure and tenant-isolation design.
- Role and permission matrix.
- Web administration portal and Electron POS terminal.
- Kitchen and bar workflow, inventory workflow and synchronization design.
- Receipt templates, operational reports and exports.
- Seed data, tests, installation guide and deployment configuration.

- Backup, recovery, security and operational support documentation.

33. Client Decisions Required Before Development

- Confirm the first target business type: restaurant/hospitality, retail, or a combined platform.
- Confirm whether the first deployment requires full offline operation.
- List the intended printers, cash drawers, scanners, terminals and operating systems.
- Confirm required payment gateways and whether payment confirmation must be automated.
- Confirm initial branches, expected users, transaction volume and reporting priorities.
- Approve the recommended TypeScript, React, Electron, Node.js and PostgreSQL architecture.
- Approve the minimum viable product scope and pilot location before full implementation.

APPROVAL

Development should proceed after the client confirms the first-release scope, target hardware, offline requirements, payment integrations and proposed technology architecture.